

MLOps & CI/CD

Disciplina/Tema: Aula 2 — Infraestrutura como Código com Terraform

Professor(a): André Luiz Braga

MBA em BI & Data Science • Carga horária da aula: 5h



Agenda de Hoje

- Fundamentos: IaC, declarativo, idempotência
- Setup do ambiente: Terraform CLI + Azure
- Terraform 100% local — 4 exercícios sem nuvem
- Fluxo de trabalho: init, plan, apply, destroy (+ prática)
- Blocos da linguagem e vocabulário do HCL
- Estado (state), multi-cloud e boas práticas
- Ferramentas de visualização e geração de HCL
- Demonstração ao vivo: infraestrutura do projeto-guia PCDF
- Exercícios práticos e depuração de erros, em cima da demo
- Estudo de caso: environment parity
- Atividade prática individual, no seu projeto

Objetivos de Aprendizagem

- **Compreender os princípios de Infraestrutura como Código (IaC)**
- **Provisionar infraestrutura em nuvem de forma declarativa com Terraform**

O Que Você Já Tem Pronto (Vindo da Aula 1)

-  Git, Python 3.11+, VS Code (+ extensão HashiCorp Terraform), Docker Desktop, Azure CLI — instalados
-  Conta Azure criada, az login já autenticado
-  Resource Group rg-pcdf-demo já existe — criado via az group create, não via Terraform
-  Repositório pcdf-bo-triagem clonado, com a pasta terraform/ vazia esperando o main.tf
-  Terraform CLI — ainda não instalado (é o nosso primeiro passo hoje)

PARTE 1

Fundamentos

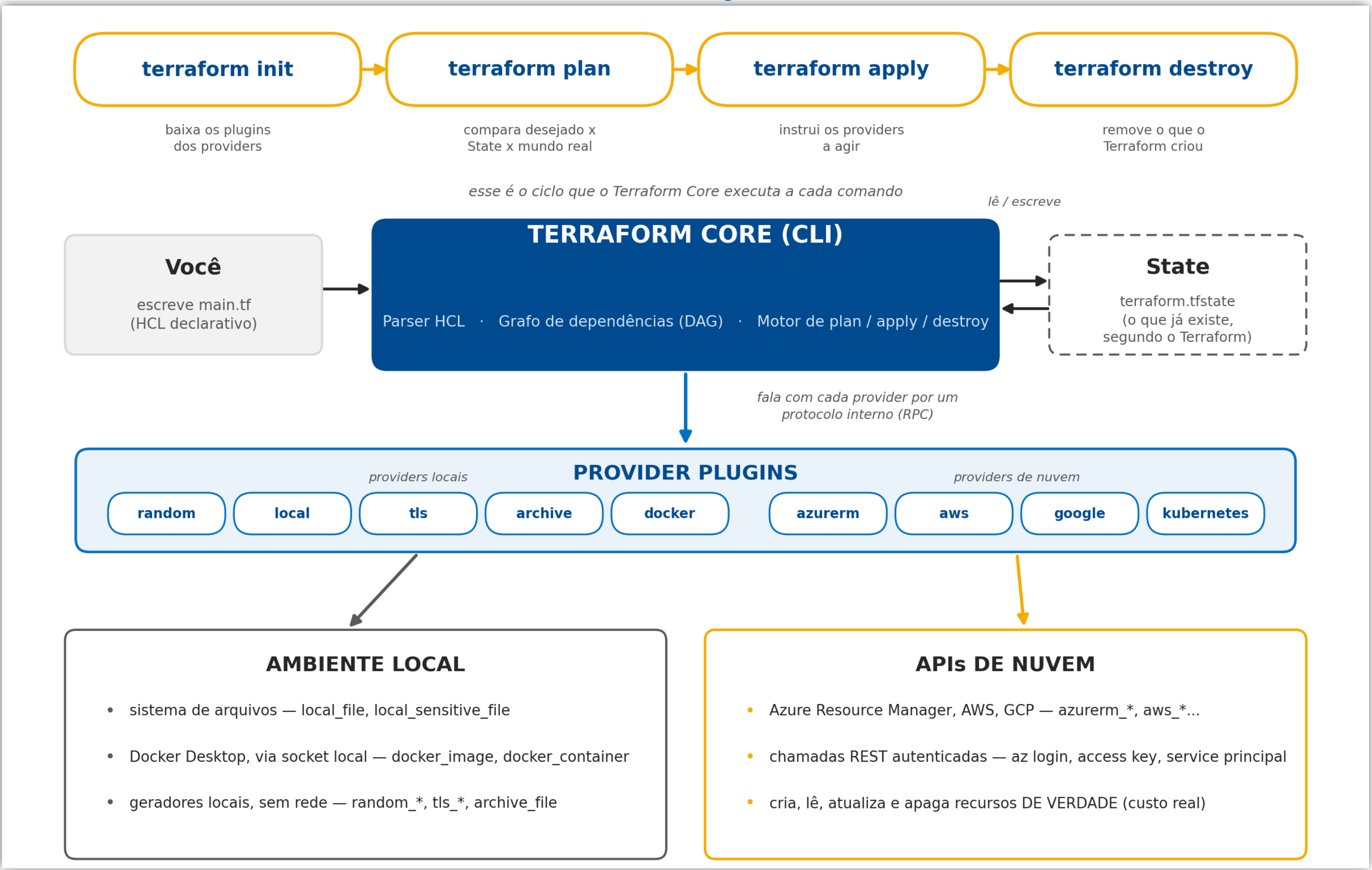
Por que Infraestrutura como Código, e o vocabulário que sustenta tudo o que vem depois.

O que é Infraestrutura como Código (IaC)?

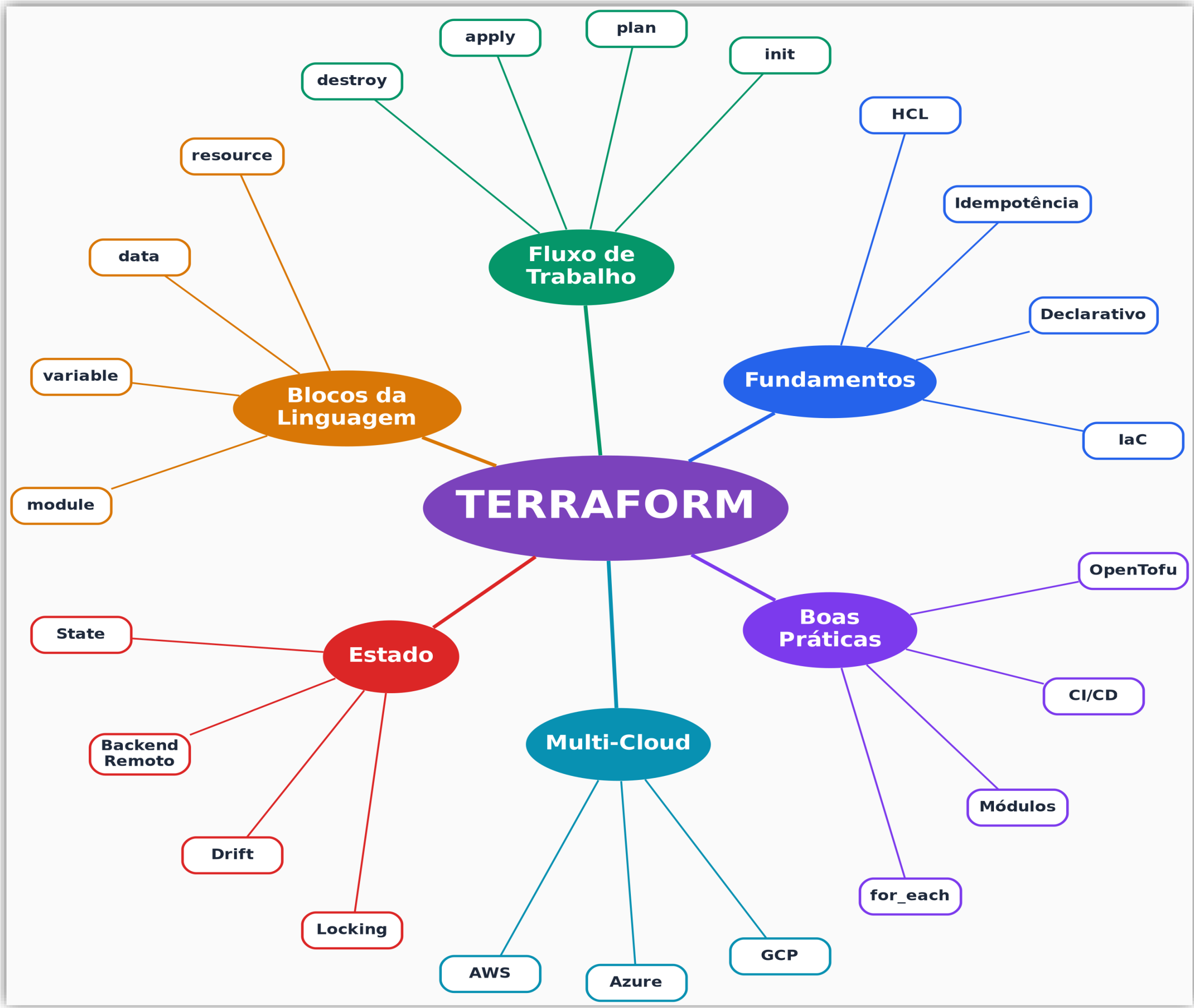
IaC: Reprodutibilidade, Versionamento, Idempotência

- **Infraestrutura como Código (IaC):** descrever infraestrutura em arquivos versionáveis, em vez de clicar no console do provedor de nuvem.
- **Reprodutibilidade:** o mesmo código gera o mesmo ambiente sempre — em qualquer máquina, por qualquer pessoa do time.
- **Versionamento:** a infraestrutura passa a viver no Git, com histórico, revisão por Pull Request e rollback, igual ao código da aplicação.
- **Idempotência:** aplicar o mesmo código Terraform várias vezes produz sempre o mesmo resultado final, sem duplicar nada — é essa garantia que torna terraform apply seguro de rodar de novo.

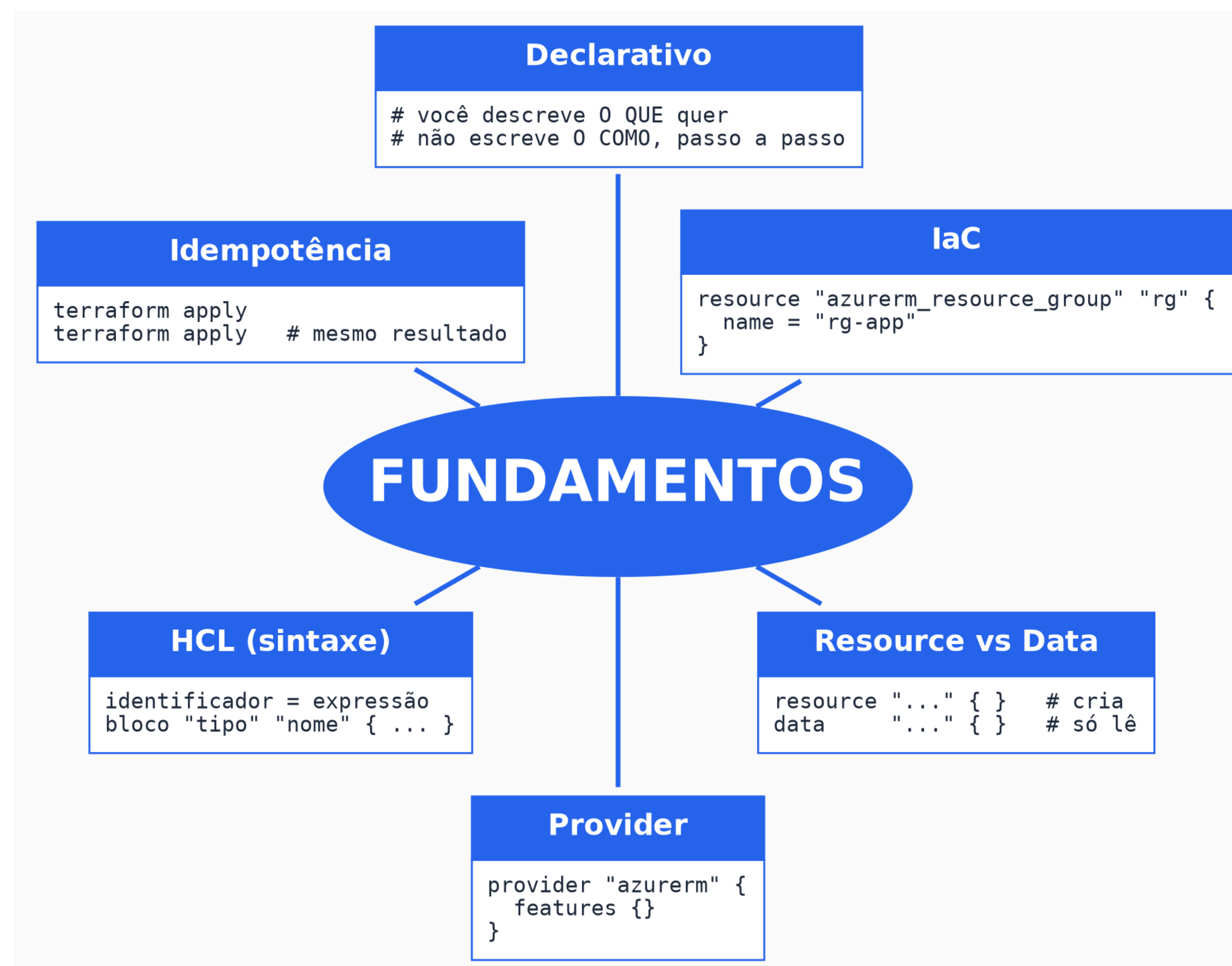
Terraform - Arquitetura



Visão Funcional do Terraform



Os Fundamentos do Terraform



Declarativo x Imperativo

- **Declarativo (Terraform):** você descreve o resultado desejado — "quero um Storage Account assim" — e o Terraform calcula sozinho os passos para chegar lá. **“Configuração”**
- **Imperativo (um script bash, por exemplo):** você escreve o passo a passo de como chegar ao resultado, na ordem exata em que deve acontecer. **“Código”**
- **Por que isso importa:** o modo declarativo é o que permite ao Terraform comparar "o que existe" com "o que o código pede" e calcular só a diferença — é a base do terraform plan.

Como Ler o Código Terraform de Hoje

Todo exemplo de código HCL de hoje segue a mesma convenção de cores — a régua mental é sempre a mesma pergunta: "isso é do Terraform, é de um provider, ou fui eu que inventei esse nome?"

● núcleo do Terraform ● vocabulário fixo (*provider/backend*) ● escolhido livremente por você ● erro proposital

- **núcleo do Terraform:** palavras que a própria linguagem reserva: `resource`, `provider`, `variable`, `count`, `var.`, entre outras.
- **vocabulário fixo do provider/backend:** nomes definidos por quem mantém o provider (`azurerm`, `aws`, `google...`) ou o tipo de backend — não são inventados por você, mas também não são do Terraform core.
- **escolhido livremente por você:** nomes de recursos, variáveis e valores — a parte que você decide ao escrever o código.
- **erro proposital:** usado só nos exercícios de depuração, para destacar onde está o problema.

Anatomia de um Arquivo Terraform

cada trecho de código pertence a uma das 6 áreas do mapa mental

FUNDAMENTOS

declarativo: você descreve o resultado, não os passos

ESTADO

o backend guarda o state remoto

MULTI-CLOUD

o provider define em qual nuvem isso roda

```
main.tf

terraform {
  required_providers {
    azurearm = {
      source = "hashicorp/azurearm"
    }
  }
  backend "azurearm" {
    container_name = "tfstate"
  }
}

provider "azurearm" {
  features {}
}

resource "azurearm_storage_account" "storage" {
  count    = 2
  name     = "st${count.index}"
  location = "Brazil South"
}

output "ids" {
  value = azurearm_storage_account.storage[*].id
}
```

BLOCOS DA LINGUAGEM

resource, variable, output, module...

BOAS PRÁTICAS

count/for_each evita repetir o mesmo bloco

FLUXO DE TRABALHO

como você aplica este arquivo de verdade

```
$ terraform init
$ terraform plan
$ terraform apply
```

PARTE 2

Setup do Ambiente

Primeiro passo prático do dia: instalar o Terraform CLI e confirmar a autenticação no Azure.

Instalar o Terraform CLI

```
# Windows:  
winget install -e --id Hashicorp.Terraform  
  
# Mac (Homebrew):  
brew tap hashicorp/tap && brew install hashicorp/tap/terraform  
  
# resultado esperado:  
terraform -version
```

Windows (Chocolatey) e Mac (Homebrew) — link:
<http://developer.hashicorp.com/terraform/downloads>

Instalar o Terraform CLI

```
Windows PowerShell
PS C:\windows\system32> winget install -e --id Hashicorp.Terraform
The 'msstore' source requires that you view the following agreements before using.
Terms of Transaction: https://aka.ms/microsoft-store-terms-of-transaction
The source requires the current machine's 2-letter geographic region to be sent to the backend service
to function properly (ex. "US").

Do you agree to all the source agreements terms?
[Y] Yes [N] No: Y
Found HashiCorp Terraform [Hashicorp.Terraform] Version 1.15.8
This application is licensed to you by its owner.
Microsoft is not responsible for, nor does it grant any licenses to, third-party packages.
Downloading https://releases.hashicorp.com/terraform/1.15.8/terraform_1.15.8_windows_amd64.zip
34.0 MB / 34.0 MB
Successfully verified installer hash
Extracting archive...
Successfully extracted archive
Starting package install...
Path environment variable modified; restart your shell to use the new value.
Command line alias added: "terraform"
Successfully installed
```

Feche e abra de novo

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\andre> terraform -version
Terraform v1.15.8
on windows_amd64
PS C:\Users\andre>
```


Instalar o Terraform CLI – A partir do zip

HashiCorp Developer Products Learn Search

Terraform Install Tutorials Documentation Sandbox Registry Try HCP Terraform

< Terraform Home

Install Terraform

Operating Systems

macOS

Windows

Linux

FreeBSD

OpenBSD

Solaris

Release information

Next steps

Resources

Tutorial Library

Certifications

Sandbox

Community Forum

Support

GitHub

Terraform Registry

Developer / Terraform

1.15.8 (latest)

Install Terraform

macOS

Package manager

```
brew tap hashicorp/tap
brew install hashicorp/tap/terraform
```

Binary download

AMD64 Version: 1.15.8	Download
ARM64 Version: 1.15.8	Download

Windows

Binary download

386 Version: 1.15.8	Download
AMD64 Version: 1.15.8	Download
ARM64 Version: 1.15.8	Download

Linux

1. Criar uma pasta fixa, por exemplo `C:\terraform`.
2. Extrair o `terraform.exe` (é só esse arquivo dentro do zip) para dentro dessa pasta.
3. Adicionar essa pasta ao PATH do Windows: menu Iniciar → pesquisar "Variáveis de Ambiente" → "Editar as variáveis de ambiente do sistema" → botão "Variáveis de Ambiente" → em "Variáveis do usuário", selecione `Path` → "Editar" → "Novo" → cole `C:\terraform` → OK em tudo.
4. Feche todos os terminais abertos e abra um novo (PowerShell ou Git Bash).
5. Teste com `terraform -version`.

Confirmar Autenticação no Azure

Pule se já executou az login recentemente (Aula 1)

```
az account show --output table
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\andre> terraform -version
Terraform v1.15.8
on windows_amd64
PS C:\Users\andre> az account show --output table
EnvironmentName      HomeTenantId          IsDefault      Name              State      TenantId
-----
AzureCloud           b73f421c-0941-4e94-bc57-25172ba3967e  True          Azure subscription 1  Enabled    b73f421c-0941-4e94-bc57-25172ba3967e
PS C:\Users\andre>
```

PARTE 3

Primeiros Passos — 100% Local, Sem Nuvem

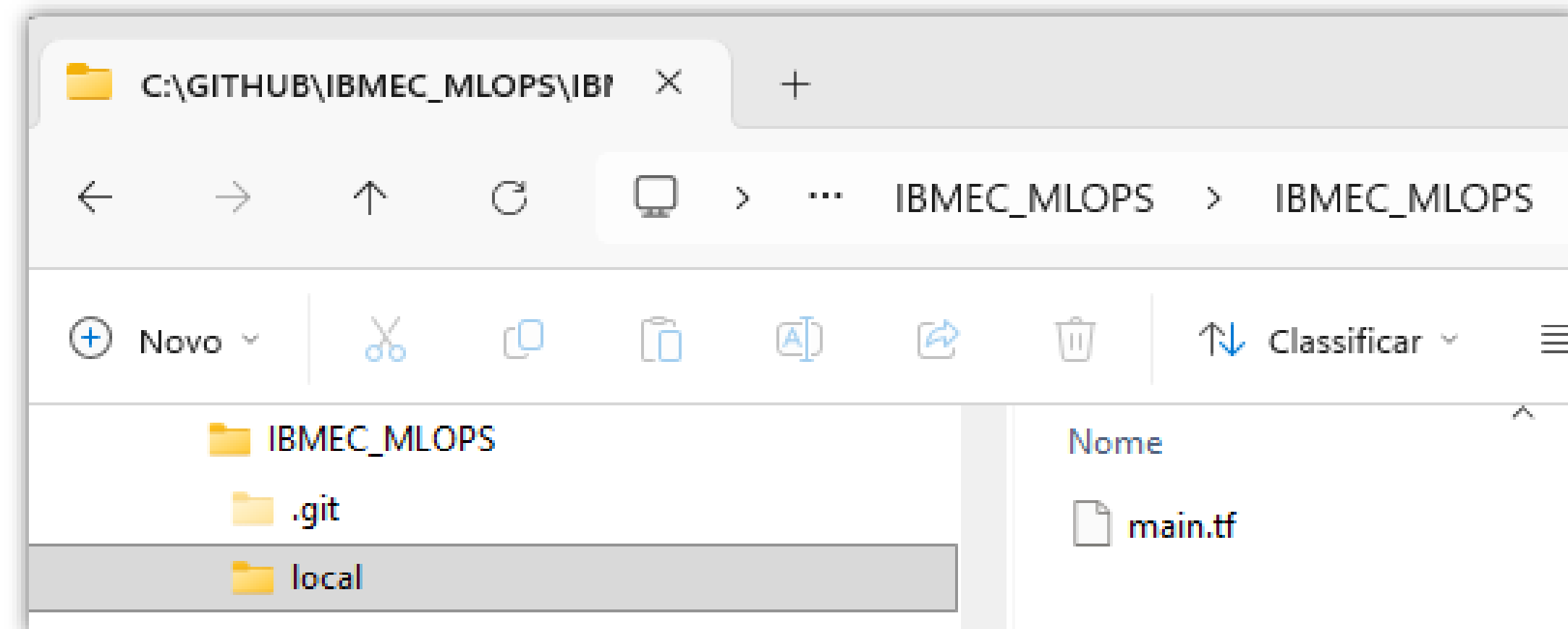
Terraform já instalado — agora vamos treinar o ciclo completo sem depender de nuvem nenhuma.

Por Que Começar Sem Nuvem?

- **Zero fricção:** sem credencial, sem conta, sem custo — só terraform init e apply.
- **Foco na sintaxe:** erra-se rápido e barato enquanto o vocabulário do HCL ainda está fresco.
- **O ciclo é o mesmo:** init → plan → apply → destroy funciona exatamente igual, seja o provider local, random, docker ou azure.
- **Progressão de hoje:** 4 exemplos locais, cada um introduzindo um provider novo, até chegarmos preparados para o Azure.

Exercício Rápido 1: Reproduza o Hello World

- **Pré-requisito:** nenhum — só o Terraform instalado (próximo passo do setup).
- **1.:** salve o código anterior em local/main.tf
- **2.:** terraform init → baixa o provider local
- **3.:** terraform apply → cria o arquivo ola.txt
- **4.:** abra ola.txt e confira o conteúdo "Olá, Terraform!"
- **Por que este exemplo primeiro:** zero custo, zero credencial de nuvem — só sintaxe e o ciclo básico.



Hello World, Anotado

o primeiro exemplo do curso, palavra por palavra

- **núcleo do Terraform**
- *vocabulário fixo (provider/backend)*
- escolhido livremente por você



```
main.tf

resource "local_file" "ola_mundo" {
  filename = "${path.module}/ola.txt"
  content  = "Olá, Terraform!"
}
```

The inset shows the VS Code interface with the Explorer pane on the left showing the file 'main.tf' under the 'LOCAL' section. The Editor pane on the right shows the content of 'main.tf' with line numbers 1 through 4.

Hello World Terraform

ibmec.br

```
Windows PowerShell
PS C:\GITHUB\IBMEC_MLOPS\IBMEC_MLOPS\local> terraform init
Initializing the backend...

Initializing provider plugins...
- Finding latest version of hashicorp/local...
- Installing hashicorp/local v2.9.0...
- Installed hashicorp/local v2.9.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\GITHUB\IBMEC_MLOPS\IBMEC_MLOPS\local>
```

```
Windows PowerShell

Terraform will perform the following actions:

# local_file.Ola_Mundo will be created
+ resource "local_file" "Ola_Mundo" {
  + content           = "Olá, Mundo Terraform!"
  + content_base64sha256 = (known after apply)
  + content_base64sha512 = (known after apply)
  + content_md5        = (known after apply)
  + content_sha1        = (known after apply)
  + content_sha256       = (known after apply)
  + content_sha512       = (known after apply)
  + directory_permission = "0777"
  + file_permission     = "0777"
  + filename           = "./ola.txt"
  + id                  = (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value:
```


Hello World Terraform

```
Windows PowerShell
PS C:\GITHUB\IBMEC_MLOPS\IBMEC_MLOPS\local> terraform apply

Terraform used the selected providers to generate the following execution
plan.
Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# local_file.Ola_Mundo will be created
+ resource "local_file" "Ola_Mundo" {
+   content          = "Olá, Mundo Terraform!"
+   content_base64sha256 = (known after apply)
+   content_base64sha512 = (known after apply)
+   content_md5       = (known after apply)
+   content_sha1       = (known after apply)
+   content_sha256     = (known after apply)
+   content_sha512     = (known after apply)
+   directory_permission = "0777"
+   file_permission    = "0777"
+   filename          = "./ola.txt"
+   id                 = (known after apply)
}

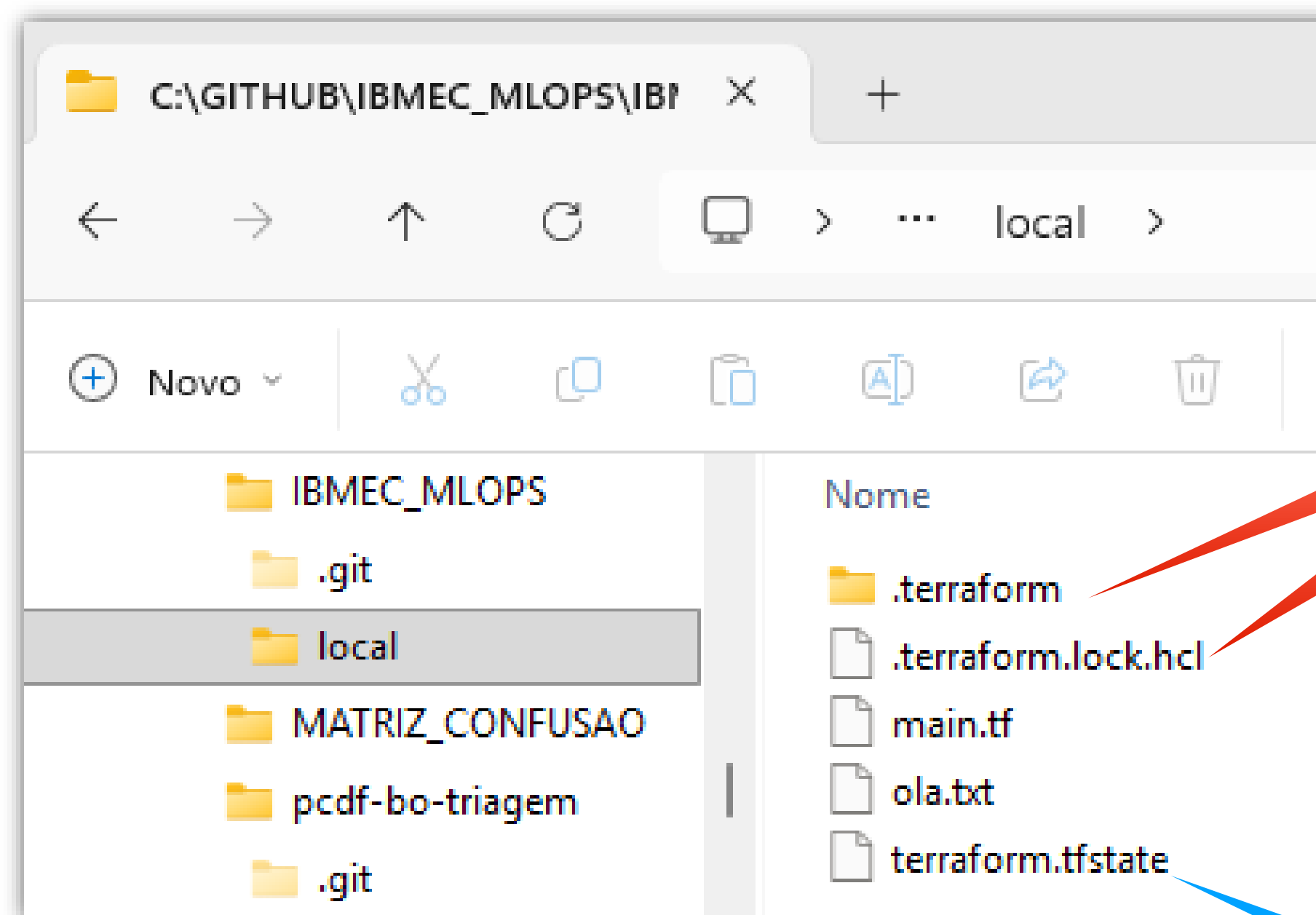
Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

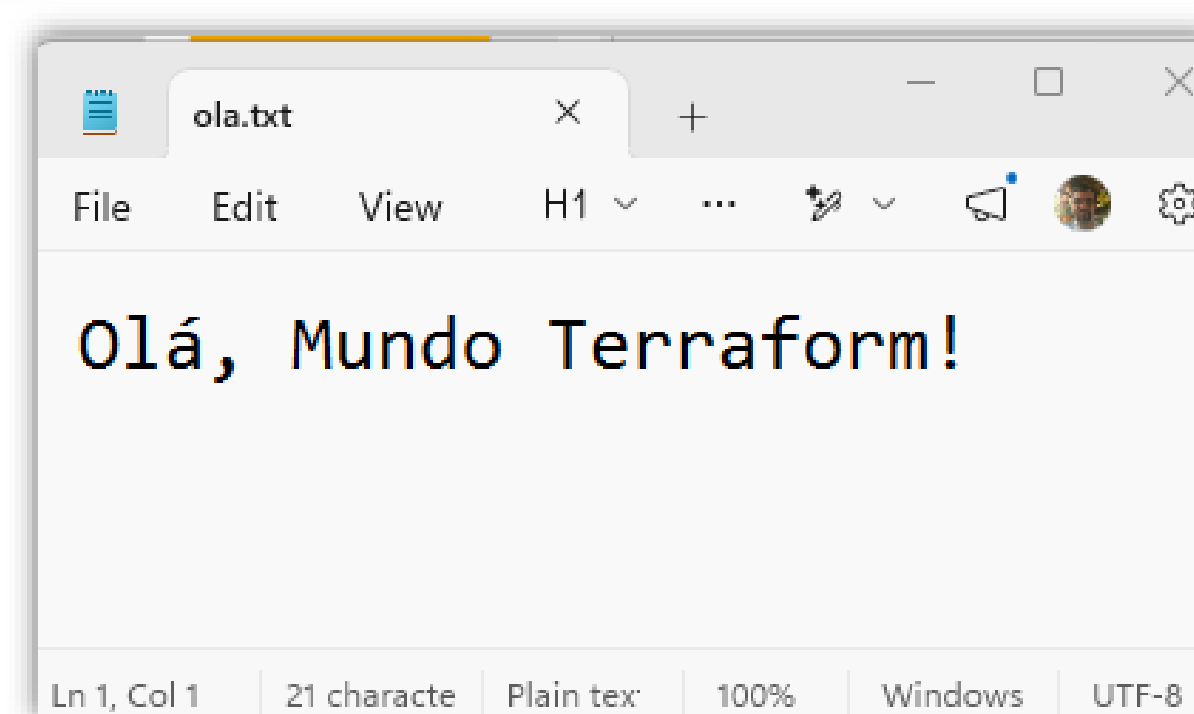
Enter a value: yes

local_file.Ola_Mundo: Creating...
local_file.Ola_Mundo: Creation complete after 0s [id=e1d7d1ffb9c14aaf72ed
68dec665c48bd465b743]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\GITHUB\IBMEC_MLOPS\IBMEC_MLOPS\local>
```



Uso Interno



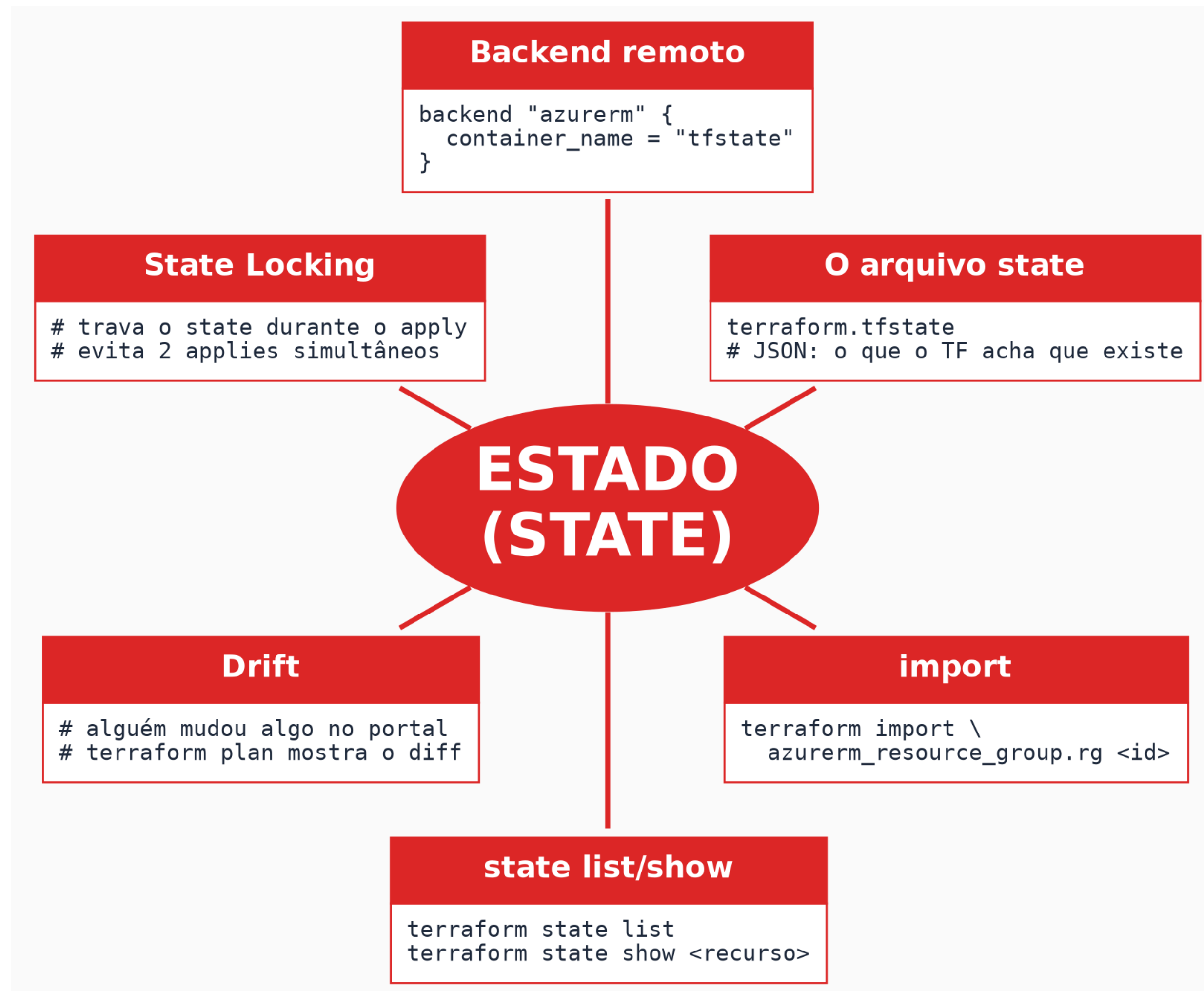
Estado para
futuras
mudanças

PARTE 4

Estado (State)

A "memória" do Terraform — e por que ela é a peça mais delicada de todo o sistema.

O Estado (State)



O Que Acontece se Eu Perder o Arquivo de State?

Resposta

- Os recursos continuam existindo de verdade na nuvem — só o Terraform perde a referência de que ele os criou.
- Para recuperar o controle, seria preciso importar cada um manualmente (terraform import ou bloco import).
- É exatamente por isso que backend remoto com versionamento (Azure Storage com versioning, S3 com versioning) é considerado essencial, não opcional.
- Hoje o state fica local (terraform.tfstate, que NÃO deve ir pro Git) — a partir do momento em que mais de uma pessoa mexe no projeto, isso migra para um backend remoto (gancho para próximas aulas).

Por Que Não Editar o terraform.tfstate na Mão?

Resposta

- É um JSON de formato interno que pode mudar entre versões do Terraform — editar manualmente arrisca corromper referências entre recursos.
- **Para ajustes, use os comandos próprios:** terraform state mv (renomear/mover) e terraform state rm (remover do controle sem destruir de verdade).
- **State Locking:** trava o state durante um apply, para dois processos não rodarem ao mesmo tempo e corromper tudo.
- **Drift:** quando alguém muda algo manualmente fora do Terraform (ex.: no portal) — o terraform plan é o jeito de descobrir isso.

OUTROS EXERCÍCIOS LOCAIS

 local

 .terraform

 Exemplo-credenciais

 exemplo-docker

 Exemplo-ssh

 exemplo-zip

 app

Exemplo 1a — Provedores Necessários

providers random e local — sem pré-requisitos de nuvem

● núcleo do Terraform ● vocabulário fixo (provider/backend) ● escolhido livremente por você

```
main1.tf

terraform {
  required_providers {
    random = {
      source = "hashicorp/random"
      version = "~> 3.0"
    }
    local = {
      source = "hashicorp/local"
      version = "~> 2.0"
    }
  }
}
```


Exemplo 1b — Gerando as Credenciais

mesmo arquivo, continuação — recursos e output

● núcleo do Terraform ● vocabulário fixo (provider/backend) ● escolhido livremente por você

```
main1.tf

resource "random_pet" "usuario" {
  length = 2
  separator = "_"
}

resource "random_password" "senha" {
  length = 16
  special = true
}

resource "local_sensitive_file" "credenciais" {
  filename = "${path.module}/credenciais.txt"
  content = "usuario: ${random_pet.usuario.id}\nsenha: ${random_password.senha.result}"
}

output "usuario_gerado" {
  value = random_pet.usuario.id
}
```

sensitive Não é Criptografia

- **Reparem:** não existe output da senha de propósito — para vê-la, é preciso abrir credenciais.txt ou usar terraform output -json explicitamente.
- **Pegadinha clássica:** sensitive = true esconde o valor do terminal no plan/apply, mas o valor continua em texto plano dentro do arquivo de state.
- **Conclusão:** sensitive evita que a senha apareça "à toa" no terminal — não substitui um cofre de segredos de verdade (Azure Key Vault, por exemplo).

Exemplo 2a — Provedores de Chaves e Arquivos

providers tls + local — gera um par de chaves criptográficas localmente

● núcleo do Terraform ● vocabulário fixo (provider/backend) ● escolhido livremente por você

```
main2.tf

terraform {
  required_providers {
    tls = {
      source = "hashicorp/tls"
      version = "~> 4.0"
    }
    local = {
      source = "hashicorp/local"
      version = "~> 2.0"
    }
  }
}
```


Exemplo 2b — Par de Chaves SSH Real

mesmo arquivo, continuação — depois do apply, teste com `ssh-keygen -lf id_rsa.pub`

● núcleo do Terraform ● vocabulário fixo (provider/backend) ● escolhido livremente por você

```
main2.tf

resource "tls_private_key" "chave" {
  algorithm = "RSA"
  rsa_bits  = 4096
}

resource "local_file" "chave_privada" {
  content = tls_private_key.chave.private_key_pem
  filename = "${path.module}/id_rsa"
  file_permission = "0600"
}

resource "local_file" "chave_publica" {
  content = tls_private_key.chave.public_key_openssh
  filename = "${path.module}/id_rsa.pub"
}

output "fingerprint" {
  value = tls_private_key.chave.public_key_fingerprint_sha256
}
```

Antes de Rodar o Exemplo 3: Crie a Pasta app/

data "archive_file" precisa que ./app já exista com algo dentro — sem isso, o apply falha

```
mkdir app
echo 'console.log("Olá do app de exemplo!");' > app/index.js

# estrutura esperada antes do apply:
# main3.tf
# app/
#   └─ index.js
```

Exemplo 3a — Provedor de Arquivos Compactados

provider archive — sem nenhuma dependência de nuvem

● núcleo do Terraform ● vocabulário fixo (provider/backend) ● escolhido livremente por você

```
main3.tf

terraform {
  required_providers {
    archive = {
      source = "hashicorp/archive"
      version = "~> 2.0"
    }
  }
}
```


Exemplo 3b — Empacotando um App em .zip

mesmo arquivo, continuação — providers archive + terraform_data nativo, com um provisioner

● núcleo do Terraform ● vocabulário fixo (provider/backend) ● escolhido livremente por você

```
main3.tf

data "archive_file" "pacote" {
  type = "zip"
  source_dir = "${path.module}/app"
  output_path = "${path.module}/app.zip"
}

resource "terraform_data" "info" {
  input = data.archive_file.pacote.output_sha

  provisioner "local-exec" {
    command = "echo Pacote gerado com sucesso"
  }
}

output "sha256_do_pacote" {
  value = data.archive_file.pacote.output_sha
}
```

Provisioners — Use com Moderação

- **O que é terraform_data:** um resource nativo do Terraform (desde a v1.4) — não precisa de required_providers, porque não vem de plugin nenhum.
- **provisioner "local-exec":** executa um comando depois que o recurso é criado — aqui, só um echo de demonstração.
- **Recomendação oficial da HashiCorp:** evitar provisioners como primeira opção. Preferência real: imagens já prontas (Packer), scripts nativos de inicialização da nuvem (user_data/cloud-init), ou configuration management.
- **Quando usar mesmo assim:** só quando nada mais serve — é a ferramenta de último recurso, não a primeira escolha.

Exemplo 4a — Provedor Docker

pré-requisito: Docker Desktop instalado e RODANDO (já instalado na Aula 1) — provider docker (kreuzwerker)

● núcleo do Terraform ● vocabulário fixo (provider/backend) ● escolhido livremente por você

```
Main4.tf

terraform {
  required_providers {
    docker = {
      source = "kreuzwerker/docker"
      version = "~> 3.0"
    }
  }
}
```

Exemplo 4b — Container Docker via Terraform (CONTINUAÇÃO)

mesmo arquivo, continuação — depois do apply, abra localhost:8080

● núcleo do Terraform ● vocabulário fixo (provider/backend) ● escolhido livremente por você

```
Main4.tf

provider "docker" {}

resource "docker_image" "nginx" {
  name = "nginx:latest"
}

resource "docker_container" "site" {
  name = "meu-site-local"
  image = docker_image.nginx.image_id

  ports {
    internal = 80
    external = 8080
  }
}

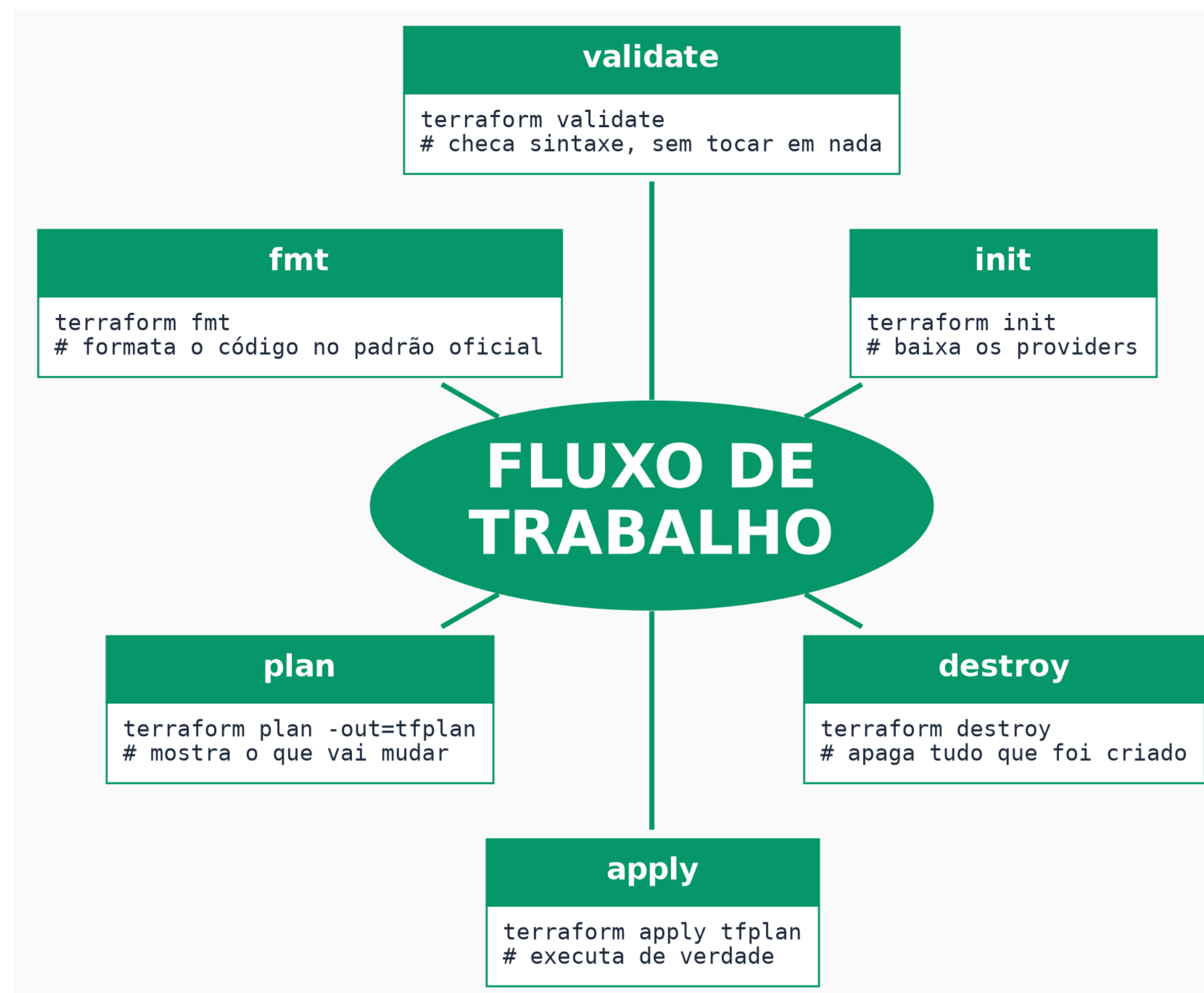
output "acesse_em" {
  value = "http://localhost:8080"
}
```

PARTE 5

Fluxo de Trabalho

Os comandos que você vai digitar toda semana, pelo resto do curso.

O Ciclo de Vida do Terraform



Comandos — Ciclo de Vida e Qualidade

Comando	O que faz
terraform init	baixa os providers/módulos declarados
terraform validate	checa sintaxe e consistência interna, sem tocar em nada
terraform fmt	formata o código no padrão oficial
terraform plan	calcula e mostra as mudanças, sem aplicar
terraform apply	executa as mudanças de verdade
terraform destroy	remove tudo que está no state
terraform refresh	atualiza o state comparando com a nuvem

Comandos — Estado e Utilitários

Comando	O que faz
terraform import	traz um recurso existente para o controle do Terraform
terraform state	subcomandos para inspecionar/editar o state (list, show, mv, rm)
terraform output	imprime os valores declarados em blocos output
terraform console	abre o REPL interativo
terraform workspace	gerencia workspaces
terraform graph	gera o grafo de dependências
terraform apply -replace	força a recriação de um recurso específico

Pratique Agora: 5 Minutos com os Comandos (opcional)

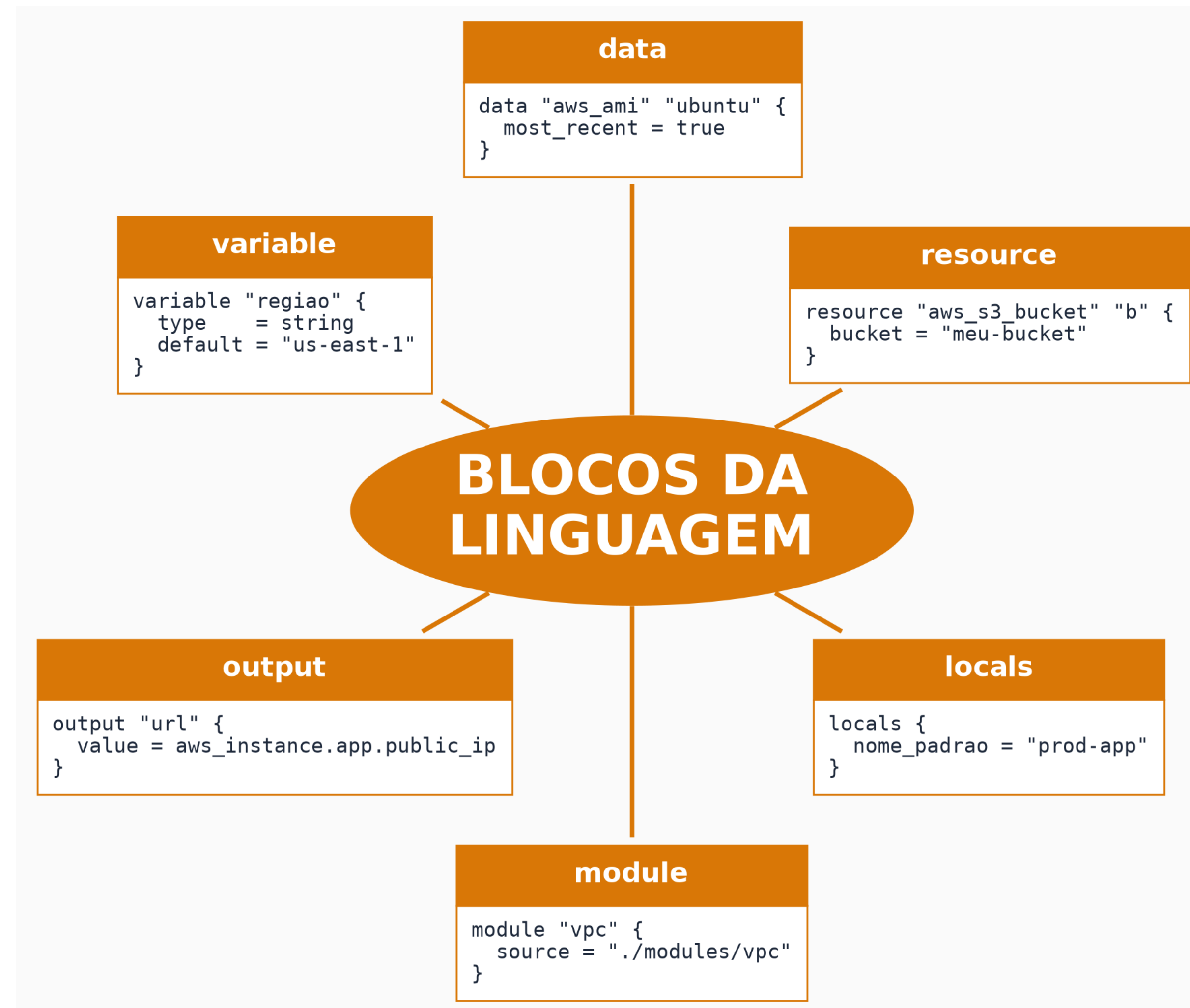
- **O que fazer:** volte a uma das pastas dos Exemplos 1 a 4 (Parte 3) e rode, na ordem: terraform plan, terraform apply, terraform output, terraform destroy.
- **Preste atenção:** o plan mostra exatamente o que vai mudar antes de qualquer coisa acontecer — compare o que ele prevê com o que o apply de fato faz.
- **Se sobrar tempo:** rode terraform state list para ver os recursos que o Terraform está controlando naquela pasta.

PARTE 6

Blocos da Linguagem

O vocabulário que o Terraform de fato reserva — e o que só parece reservado.

Os Blocos da Linguagem HCL



Prefixos de Referência (o "espelho" de cada bloco)

Prefixo	Referencia	Exemplo
var.	um variable	var.regiao
local.	um locals (singular, mesmo o bloco sendo plural)	local.nome_padrao
module.	o output de um module	module.vpc.id
data.	um data	data.aws_ami.ubuntu.id
path.	caminhos especiais (não vem de bloco nenhum)	path.module, path.root

Meta-argumentos: count, for_each, depends_on, lifecycle

- **Meta-argumentos:** não são resources — são argumentos especiais que mudam o comportamento de qualquer resource ou module.
- **count:** cria N instâncias, indexadas por número. Frágil: se um item sai do meio de uma lista, os índices deslocam e o Terraform pode destruir/recrutar a coisa errada.
- **for_each:** cria uma instância por item de um mapa/set, com chaves estáveis — é o padrão recomendado hoje para listas que podem mudar.
- **depends_on:** força uma dependência explícita, quando o Terraform não consegue inferir a ordem sozinho.
- **lifecycle:** bloco aninhado com regras especiais: create_before_destroy, prevent_destroy, ignore_changes.

Resource Group, Anotado

primeiro contato de verdade com a nuvem

● **núcleo do Terraform** ● *vocabulário fixo (provider/backend)* ● escolhido livremente por você

```
main.tf

provider "azurerm" {
  features {}
}

resource "azurerm_resource_group" "rg" {
  name      = "rg-pedidos"
  location  = "Brazil South"
}
```

Exemplo Completo, Anotado

terraform{} + provider + resource + output juntos

● núcleo do Terraform ● vocabulário fixo (provider/backend) ● escolhido livremente por você

```
main.tf

terraform {
  required_providers {
    azurerm = {
      source = "hashicorp/azurerm"
    }
  }
  backend "azurerm" {
    container_name = "tfstate"
  }
}

provider "azurerm" {
  features {}
}

resource "azurerm_storage_account" "storage" {
  count     = 2
  name      = "st${count.index}"
  location  = "Brazil South"
}

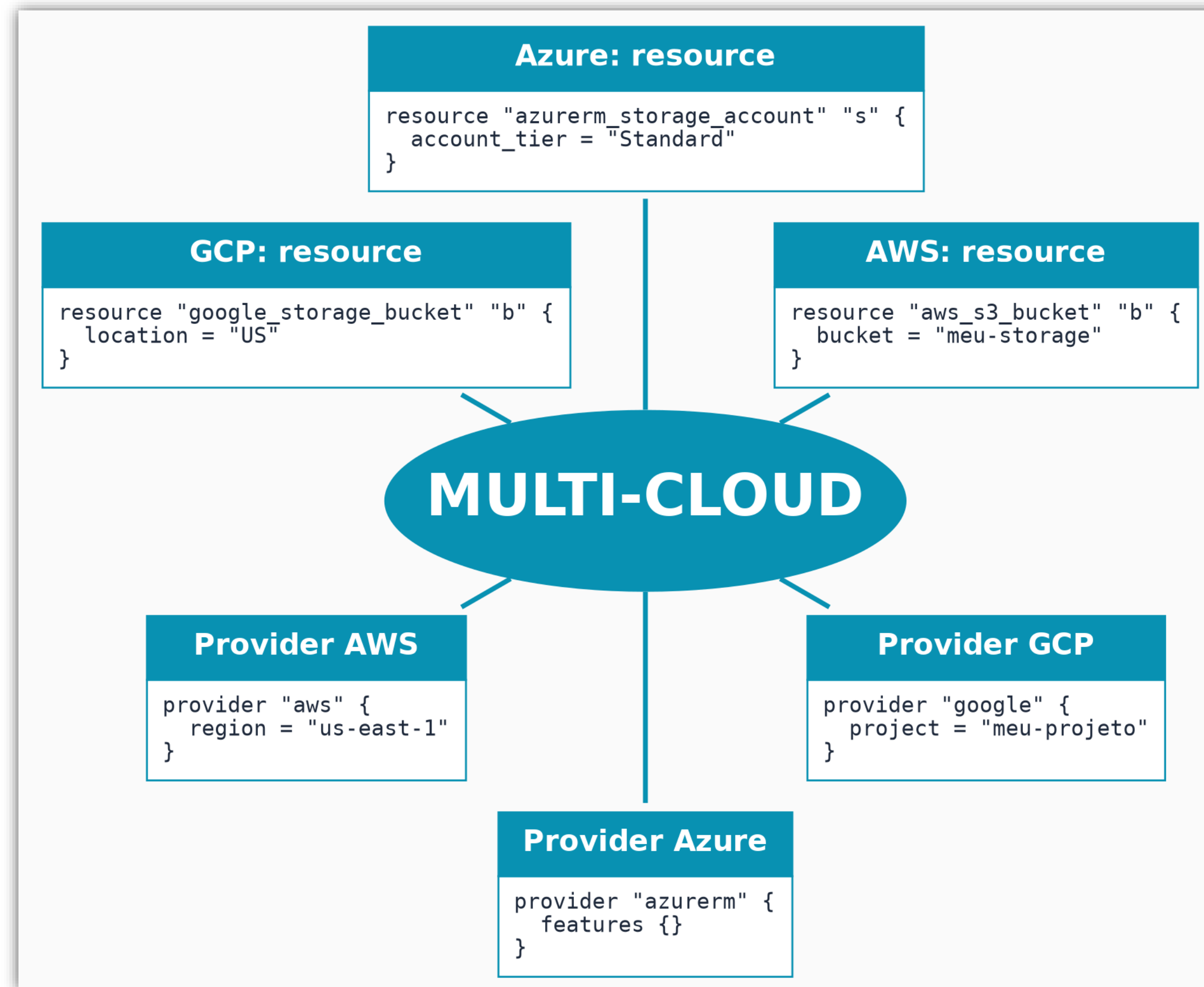
output "ids" {
  value = azurerm_storage_account.storage[*].id
}
```

PARTE 7

Multi-Cloud

Uma ferramenta só para aprender — não um código genérico universal.

Multi-Cloud



Multi-Cloud, Anotado

repare: "aws", "azurerm", "google" e os tipos de resource são todos *itálico* — nada disso é reservado pelo Terraform

● núcleo do Terraform ● vocabulário fixo (provider/backend) ● escolhido livremente por você

```
main.tf

provider "aws" {
  region = "us-east-1"
}

resource "aws_s3_bucket" "b" {
  bucket = "meu-bucket"
}

provider "azurerm" {
  features {}
}

resource "azurerm_storage_account" "s" {
  account_tier = "Standard"
}

provider "google" {
  project = "meu-projeto"
}

resource "google_storage_bucket" "b" {
  location = "US"
}
```


***Terraform é Multi-Cloud — Dá pra Migrar de Azure
pra AWS só Trocando o Provider?***

Resposta

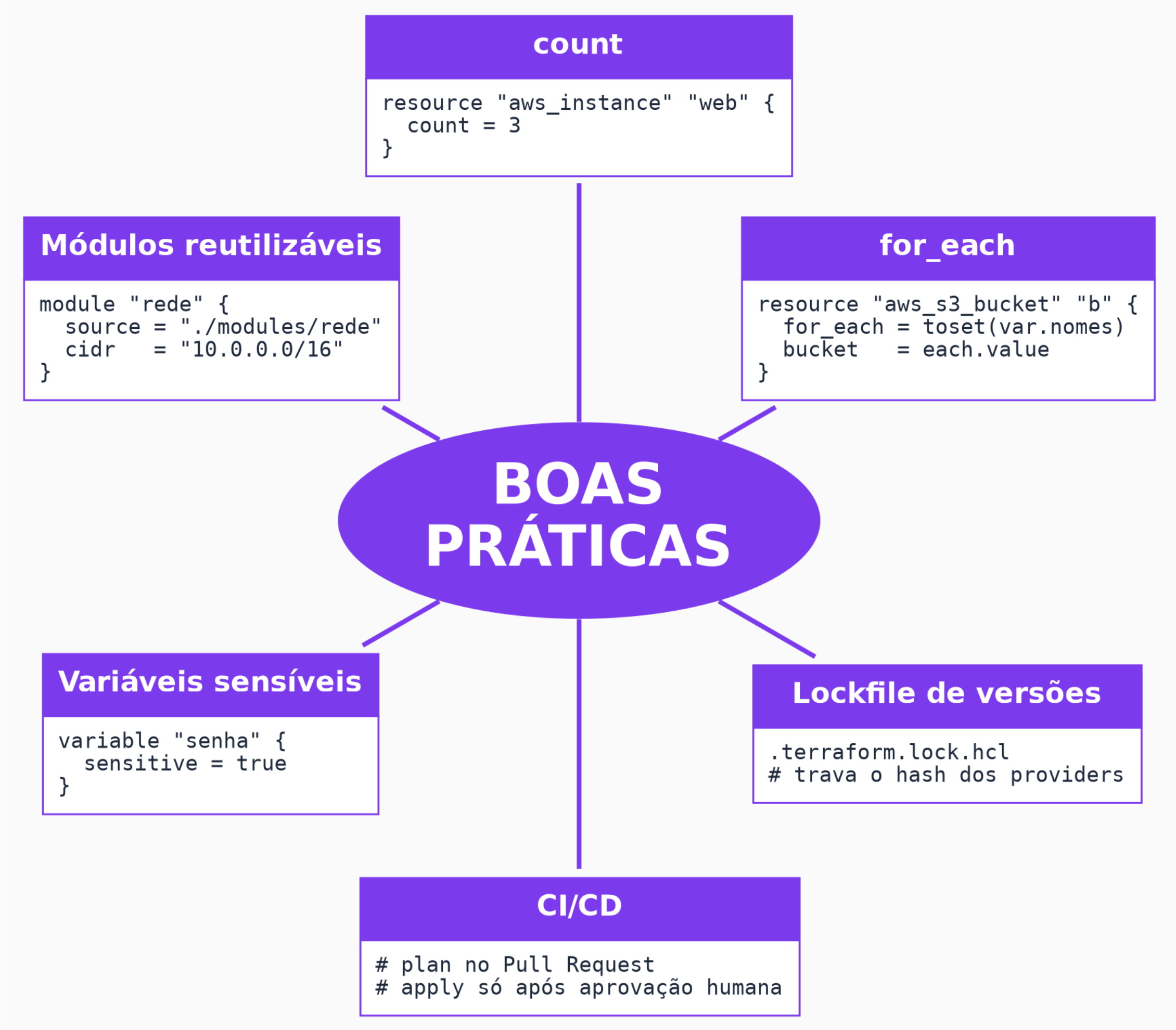
- Não automaticamente.
- A linguagem (HCL) e os conceitos (resource, state, plan/apply) são os mesmos, mas cada provider expõe tipos de resource com nomes e atributos diferentes.
- `azurerm_storage_account` não é `aws_s3_bucket` com um find-and-replace.
- "Multi-cloud" aqui significa "uma ferramenta só para aprender", não "código genérico universal".

PARTE 8

Boas Práticas

O que separa um código
Terraform de estudo de
um código pronto para
produção.

Boas Práticas



Segurança: State e Privilégio Mínimo

- **State como superfície de risco:** nunca commitar terraform.tfstate no Git — ele pode conter segredos em texto plano, mesmo os marcados sensitive.
- **Least privilege:** a credencial que o Terraform usa para falar com a nuvem deveria ter só as permissões necessárias, não acesso de administrador total.
- **Lockfile de versões:** .terraform.lock.hcl trava o hash dos providers — garante que todo mundo do time baixa exatamente a mesma versão.
- **CI/CD:** plan roda automaticamente no Pull Request; apply só acontece após aprovação humana — nunca automático em produção sem revisão.

Site com Formulário + Banco, Anotado

App Service roda o Node.js (front + back juntos) — Postgres guarda os elementos

● núcleo do Terraform ● vocabulário fixo (provider/backend) ● escolhido livremente por você

```
main.tf

terraform {
  required_providers {
    azurerm = {
      source = "hashicorp/azurerm"
    }
  }
}

provider "azurerm" {
  features {}
}

resource "azurerm_resource_group" "rg" {
  name      = "rg-elementos"
  location  = "Brazil South"
}

resource "azurerm_service_plan" "plan" {
  name                = "plan-elementos"
  resource_group_name = azurerm_resource_group.rg.name
  location            = azurerm_resource_group.rg.location
  os_type             = "Linux"
  sku_name            = "B1"
}
```

```
resource "azurerm_postgresql_flexible_server" "db" {
  name                = "elementos-db"
  resource_group_name = azurerm_resource_group.rg.name
  location            = azurerm_resource_group.rg.location
  version             = "16"
  administrator_login  = var.db_username
  administrator_password = var.db_password
  sku_name            = "B_Standard_B1ms"
}

resource "azurerm_postgresql_flexible_server_database" "elementos" {
  name      = "elementos"
  server_id = azurerm_postgresql_flexible_server.db.id
}

resource "azurerm_linux_web_app" "app" {
  name                = "elementos-app"
  resource_group_name = azurerm_resource_group.rg.name
  location            = azurerm_service_plan.plan.location
  service_plan_id     = azurerm_service_plan.plan.id

  site_config {
    application_stack {
      node_version = "20-lts"
    }
  }

  app_settings = {
    DB_HOST = azurerm_postgresql_flexible_server.db.fqdn
    DB_NAME = "elementos"
  }
}
```


PARTE 9

Ecosystema

O contexto de mercado ao redor do Terraform — o que vale saber citar.

Terraform no Contexto do Mercado

- **OpenTofu:** fork open-source do Terraform, criado pela Linux Foundation depois que a HashiCorp mudou a licença do Terraform (de MPL para BUSL) em 2023.
- **Terraform x Pulumi:** Pulumi usa linguagens de programação de verdade (Python, TypeScript) em vez de HCL.
- **Terraform x CloudFormation/Bicep:** essas são "nativas" de uma única nuvem (AWS/Azure); Terraform é multi-provider.
- **Terraform x Ansible/Chef/Puppet:** Ansible é configuration management (o que roda dentro da máquina); Terraform é provisionamento (a máquina/recurso em si). Na prática, muita gente usa os dois juntos.

PARTE 10

Ferramentas de Visualização e Geração de HCL

Terceiros que ajudam a ir de desenho para código, ou de código para diagrama.

Desenho → Código (montar HCL do zero)

- **Structura.io:** a interface de arrastar e soltar escreve o código Terraform em tempo real conforme você mexe nos campos — vendida como forma de aprender a sintaxe HCL do zero, não só gerar código para produção.
- **Brainboard:** canvas de arquitetura com drag & drop; também consegue ler um repositório Terraform existente e desenhar o diagrama de volta — funciona nos dois sentidos.
- **CloudForge:** designer multi-cloud (Azure, AWS e GCP) que gera Terraform, Pulumi, ARM, Bicep ou CloudFormation a partir do mesmo desenho.
- **Terraforge (open-source):** editor tipo grafo de nós, exporta um .tf real, mas ainda não executa comandos Terraform — o código sai sem formatação nem validação, sempre rodar terraform fmt/validate depois.

Código → Diagrama (visualizar o que já existe)

- **Inframap, Rover e Terraform Visual:** leem sua configuração, state ou plano Terraform e transformam isso em diagramas de infraestrutura.
- **terraform graph:** o comando nativo que já vimos no fluxo de trabalho — gera o grafo de dependências; essas ferramentas complementam ele com uma UI melhor.

Aviso: Nenhuma é Oficial da HashiCorp

- Nem a própria HCP Terraform tem um "construtor visual" nativo de blocos.
- São todas ferramentas de terceiros — qualidade e manutenção variam bastante.
- O HCL gerado por qualquer uma delas sempre merece revisão antes do apply.

PARTE 11

Demonstração: Provisionando a Infra do Projeto-Guia PCDF

Agora sim: o primeiro contato com o Azure de verdade, no projeto-guia.

Recapitulando o Que Já Existe

- **Resource Group rg-pcdf-demo:** já existe — criado manualmente na Aula 1 via az group create, não pelo Terraform.
- **Consequência direta:** o Terraform de hoje vai LER esse recurso com data, não CRIAR com resource — gancho natural para reforçar a diferença entre os dois.
- **O que vamos criar de fato:** uma Storage Account e um container, para guardar os BOs do projeto-guia.

Aula 02 — Preparacao do Projeto

Storage Account + container dos BOs, no rg-pcdf-demo já existente (Aula 1)

● núcleo do Terraform ● vocabulário fixo (provider/backend) ● escolhido livremente por você

```
main.tf

terraform {
  required_providers {
    azurerm = { source = "hashicorp/azurerm", version = "~> 3.0" }
  }
}

provider "azurerm" {
  features {}
}

data "azurerm_resource_group" "rg" {
  name = "rg-pcdf-demo"
}

resource "azurerm_storage_account" "st" {
  name                        = "stpcdfbodem"
  resource_group_name        = data.azurerm_resource_group.rg.name
  location                   = data.azurerm_resource_group.rg.location
  account_tier                = "Standard"
  account_replication_type    = "LRS"
}

resource "azurerm_storage_container" "raw" {
  name                        = "bos-raw"
  storage_account_name        = azurerm_storage_account.st.name
  container_access_type       = "private"
}
```

```
PS C:\GITHUB\IBMEC_MLOPS\pcdf-bo-triagem\terraform> terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/azurerm versions matching "~> 3.0"...
- Installing hashicorp/azurerm v3.117.1...
- Installed hashicorp/azurerm v3.117.1 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\GITHUB\IBMEC_MLOPS\pcdf-bo-triagem\terraform>
```

```
azurerm_storage_account.st: Still creating... [00m40s elapsed]
azurerm_storage_account.st: Still creating... [00m50s elapsed]
azurerm_storage_account.st: Still creating... [01m00s elapsed]
azurerm_storage_account.st: Creation complete after 1m5s [id=/subscriptions/8c792235-b538-499c-8a9c-10893b8ae201/resourceGroups/rg-pcdf-demo/providers/Microsoft.Storage/storageAccounts/pcdfbodem]
azurerm_storage_container.raw: Creating...
azurerm_storage_container.raw: Creation complete after 0s [id=https://pcdfbodem.blob.core.windows.net/bos-raw]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
PS C:\GITHUB\IBMEC_MLOPS\pcdf-bo-triagem\terraform> |
```


Passo a Passo, com Resultado Esperado

- **terraform init:** "Terraform has been successfully initialized!"
- **terraform plan:** "Plan: 2 to add, 0 to change, 0 to destroy." — só a Storage Account e o container contam; o data nunca aparece no plano de mudanças.
- **terraform apply -auto-approve:** recursos criados; outputs exibidos no terminal.
- **Conferir no Portal Azure:** portal.azure.com → Resource Groups → rg-pcdf-demo → Storage Account "stpcdfbodemmo" e container "bos-raw" visíveis.
- **Commit:** git add terraform/ → git commit -m "infra: storage account via terraform" → git push.

Passo a Passo, com Resultado Esperado

- **terraform plan:** "Plan: 2 to add, 0 to change, 0 to destroy." — só a Storage Account e o container contam; o data nunca aparece no plano de mudanças.

```
PS C:\GITHUB\IBMEC_MLOPS\pcdf-bo-triagem\terraform> terraform plan
data.azurem_resource_group.rg: Reading...
data.azurem_resource_group.rg: Read complete after 0s [id=/subscriptions/8c792235-b538-499c-8a9c-10893b8ae201/resourceGroups/rg-pcdf-demo]
azurem_storage_account.st: Refreshing state... [id=/subscriptions/8c792235-b538-499c-8a9c-10893b8ae201/resourceGroups/rg-pcdf-demo/providers/Microsoft.Storage/storageAccounts/pcdfbodemo]
azurem_storage_container.raw: Refreshing state... [id=https://pcdfbodemo.blob.core.windows.net/bos-raw]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.
PS C:\GITHUB\IBMEC_MLOPS\pcdf-bo-triagem\terraform> |
```

Passo a Passo, com Resultado Esperado

The screenshot shows the Microsoft Azure portal interface. The browser tabs at the top include 'Tutorial Library | HashiCorp Dev', 'Azure for Students | Microsoft A', and 'rg-pcdf-demo - Microsoft Azure'. The address bar shows the URL 'portal.azure.com/#@andreluizbraga2002gmail.onmicrosoft.com/resource/subscriptions/8c792235-b538-499c-8a9c-1089'. The main header features the 'Microsoft Azure' logo, an 'Atualizar' button, a search bar, and a 'Copilot' button. The user's profile 'andre.luiz.braga2002@...' is visible in the top right.

The left sidebar contains navigation options: 'Criar um recurso', 'Página inicial', 'Painel', 'Todos os serviços', and a 'FAVORITOS' section with 'Modelos de Aplicativo', 'Todos os recursos', 'Grupos de Recursos', 'Serviços de Aplicativos', 'Aplicativo de Funções', and 'Banco de Dados SQL do Azure'.

The main content area shows the 'rg-pcdf-demo' resource group. The 'Visão geral' tab is selected, displaying a list of resources. The table below shows the resource 'pcdfbodemo'.

Nome ↑	Tipo	Localização
pcdfbodemo	Conta de armazenamento	Brazil South

At the bottom of the table, it says 'Mostrando 1 - 1 de 1. Contagem de exibição: au...'. There is also a link to 'Enviar comentários'.

Provisione a Infraestrutura Básica do Seu Projeto

- **O que fazer:** criar um terraform/main.tf no repositório do SEU projeto individual, provisionando ao menos uma Storage Account via Terraform.
- **Reaproveite o padrão de hoje:** terraform {} + provider "azurerm" + data para o resource group (se já existir) + resource para a Storage Account.
- **Rode o ciclo completo:** terraform init → terraform plan → terraform apply.
- **Ao final:** confira no Portal Azure e faça commit do código terraform/ no seu repositório.

ENCERRAMENTO

**Revisão e Entregável de Hoje
(Pra semana que vem)**

Checklist de Encerramento

- ☐ Terraform CLI instalado
- ☐ az account show confirmado
- ☐ Demo ao vivo rodada e aplicada (main.tf do projeto-guia)
- ☐ Código terraform/ commitado no repositório do PROJETO INDIVIDUAL de cada aluno
- ☐ .gitignore cobrindo .terraform/ e terraform.tfstate* — é comum esquecer e subir o state (que pode ter dados sensíveis) para o Git

Entregável de Hoje

- **O que entregar:** o arquivo terraform/main.tf do seu projeto individual, committado e com push feito para o repositório no GitHub.
- **Mensagem de commit sugerida:** infra: storage account via terraform
- **Antes de subir:** confirme que .terraform/ e terraform.tfstate* estão no .gitignore — nunca commitar o state.

Referências

- **Livro:** Morris, K. (2025). Infrastructure as Code, 3ª ed. O'Reilly Media.
- **Terraform — What is Infrastructure as Code:**
developer.hashicorp.com/terraform/intro
- **Terraform: Build Infrastructure (Azure):**
developer.hashicorp.com/terraform/tutorials/azure-get-started
- **Provider azurearm:**
registry.terraform.io/providers/hashicorp/azurerm/latest/docs



IBMEC.BR

 /IBMEC

 IBMEC

 @IBMEC_OFICIAL

 @IBMEC

 **ibmec**